PROJECT REPORT

ON

CRYPTOGRAPHIC ENCRYPTION VAULT

A Secure File Storage System Using Password-Based Encryption

Submitted by

Name: _____

Roll Number: _____

Department: _____

College: _____

Submitted in partial fulfilment of the requirements for the award of the degree

Academic Year: 2026–2027

CERTIFICATE

This is to certify that the project report entitled “Cryptographic Encryption Vault” is a record of project work prepared by _____, bearing Roll Number _____, under the guidance of _____. The work is submitted in partial fulfilment of the academic requirements of the institution.

Project Guide: _____ Head of Department: _____

Date: _____ Place: _____

ABSTRACT

The Cryptographic Encryption Vault is a software application designed to protect confidential files by encrypting them before storage. The application uses password-based key derivation and authenticated encryption so that stored content cannot be read without the correct password and tampering can be detected during decryption. The proposed implementation uses Python and the cryptography library. A password is processed using a key-derivation function with a random salt to generate an encryption key. The file is then encrypted using AES-GCM, which provides confidentiality and integrity. The encrypted output contains the salt, nonce, and ciphertext required for later decryption. The project demonstrates important cybersecurity concepts such as encryption, decryption, key derivation, authentication, and secure file handling. The system is intended as an educational prototype; production deployment would require additional security review, backup procedures, access controls, and careful protection of passwords and keys.

Keywords

Cryptography, AES-GCM, encryption vault, password-based key derivation, data confidentiality, integrity.

TABLE OF CONTENTS

1. Introduction

2. Problem Statement

3. Objectives

4. Existing and Proposed Systems

5. System Requirements

6. System Architecture

7. Methodology

8. Algorithm

9. Implementation

10. Testing and Results

11. Advantages and Limitations

12. Applications

13. Conclusion and Future Scope

14. References

1. INTRODUCTION

Digital documents often contain personal, academic, financial, or organizational information. If files are stored in plain text, anyone who gains access to the storage location may be able to read them. Cryptography addresses this risk by transforming readable data into ciphertext using a cryptographic algorithm and a secret key. An encryption vault provides a controlled way to encrypt files before storing or sharing them.

This project develops a simple file-vault prototype in Python. The user supplies a password, and the application derives a cryptographic key from it. The file is encrypted locally, and only the encrypted output is saved. During decryption, the same password is used to derive the key and authenticate the encrypted content.

2. PROBLEM STATEMENT

The objective is to reduce the risk of unauthorized disclosure of confidential files stored on a computer. The system should encrypt a selected file, require a password to decrypt it, and detect incorrect passwords or modified ciphertext. It should avoid storing the password directly in the output file.

3. OBJECTIVES

• To build a simple application for encrypting and decrypting files.

• To derive encryption keys from user passwords using a salt and a key-derivation function.

• To use authenticated encryption to protect confidentiality and detect tampering.

• To demonstrate safe handling of encryption metadata such as salt and nonce.

• To test successful decryption, incorrect passwords, and modified encrypted files.

4. EXISTING AND PROPOSED SYSTEMS

4.1 Existing System

Files are commonly stored in folders without application-level encryption. Operating-system permissions may restrict access, but a copied file or an exposed storage device can still reveal its contents if the file itself is not encrypted.

4.2 Proposed System

The proposed vault encrypts each file using AES-GCM. A random salt is used to derive a key from the user's password, and a random nonce is generated for each encryption operation. AES-GCM authenticates the encrypted content; incorrect passwords or altered data cause decryption to fail.

5. SYSTEM REQUIREMENTS

Hardware: A standard desktop or laptop with sufficient storage for the input and encrypted files.

Software: Python 3.10 or later, the cryptography package, and a terminal or code editor.

Installation command: pip install cryptography

6. SYSTEM ARCHITECTURE

• User selects a file and enters a password.

• The application generates a random salt and derives a key from the password.

• AES-GCM encrypts the file using the derived key and a fresh nonce.

• The application writes a version marker, salt, nonce, and ciphertext to the vault file.

• For decryption, the application reads the metadata, derives the key again, and authenticates/decrypts the ciphertext.

• If authentication fails, the application reports failure and does not produce plaintext output.

7. METHODOLOGY

7.1 Password-Based Key Derivation

A password is not used directly as an AES key. Scrypt derives a fixed-length key from the password and a randomly generated salt. The salt is not secret; its purpose is to make identical passwords produce different derived keys and to hinder precomputed attacks. The chosen Scrypt parameters should be tested on the target hardware and adjusted according to security requirements.

7.2 AES-GCM Encryption

AES-GCM is an authenticated-encryption mode. It provides confidentiality and an authentication tag that detects modifications. A unique nonce must be used for every encryption under the same key. This implementation generates a new random nonce and salt for each encrypted file.

7.3 Decryption

The vault reads the salt and nonce from the encrypted file, derives the key from the password, and attempts authenticated decryption. If the password is incorrect or the encrypted bytes have been modified, authentication fails.

8. ALGORITHM

• Start the program and select encrypt or decrypt.

• For encryption, read the source file and request a password.

• Generate a random salt and nonce.

• Derive a 256-bit key using Scrypt.

• Encrypt the file contents using AES-GCM and save the metadata plus ciphertext.

• For decryption, parse the metadata and request the password.

• Derive the key and authenticate/decrypt the ciphertext.

• Save plaintext only after successful authentication; otherwise display an error.

• Stop.

9. IMPLEMENTATION

Save the following code as vault.py. It provides command-line encryption and decryption for educational use. The encrypted file format starts with a version marker, followed by a 16-byte salt, a 12-byte nonce, and the AES-GCM ciphertext including its authentication tag.

```
from pathlib import Path
import argparse
import getpass
import os

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt

MAGIC = b"CEV1"
SALT_LEN = 16
NONCE_LEN = 12

def derive_key(password: str, salt: bytes) -> bytes:
    kdf = Scrypt(
        salt=salt,
        length=32,
        n=2**14,
        r=8,
        p=1,
    )
    return kdf.derive(password.encode("utf-8"))

def encrypt_file(source: Path, destination: Path) -> None:
    password = getpass.getpass("Vault password: ")
    if not password:
        raise ValueError("Password cannot be empty.")
    plaintext = source.read_bytes()
    salt = os.urandom(SALT_LEN)
    nonce = os.urandom(NONCE_LEN)
    key = derive_key(password, salt)
    ciphertext = AESGCM(key).encrypt(nonce, plaintext, MAGIC)
    destination.write_bytes(MAGIC + salt + nonce + ciphertext)
    print(f"Encrypted file saved to: {destination}")

def decrypt_file(source: Path, destination: Path) -> None:
    password = getpass.getpass("Vault password: ")
    data = source.read_bytes()
    minimum = len(MAGIC) + SALT_LEN + NONCE_LEN + 16
    if len(data) < minimum or not data.startswith(MAGIC):
        raise ValueError("Invalid or unsupported vault file.")
    offset = len(MAGIC)
    salt = data[offset:offset + SALT_LEN]
    offset += SALT_LEN
    nonce = data[offset:offset + NONCE_LEN]
    offset += NONCE_LEN
    ciphertext = data[offset:]
    key = derive_key(password, salt)
    try:
        plaintext = AESGCM(key).decrypt(nonce, ciphertext, MAGIC)
    except InvalidTag:
        raise ValueError("Wrong password or the vault file was modified.")
    destination.write_bytes(plaintext)
```

```
    print(f"Decrypted file saved to: {destination}")

def main():
    parser = argparse.ArgumentParser(
        description="Educational Cryptographic Encryption Vault"
    )
    parser.add_argument("mode", choices=["encrypt", "decrypt"])
    parser.add_argument("source", type=Path)
    parser.add_argument("destination", type=Path)
    args = parser.parse_args()

    if not args.source.is_file():
        parser.error("Source file does not exist.")
    if args.source.resolve() == args.destination.resolve():
        parser.error("Source and destination must be different files.")

    if args.mode == "encrypt":
        encrypt_file(args.source, args.destination)
    else:
        decrypt_file(args.source, args.destination)

if __name__ == "__main__":
    main()
```

9.1 How to Run

• Install Python and the dependency: pip install cryptography

• Create a text file named secret.txt with sample content.

• Encrypt: python vault.py encrypt secret.txt secret.cev

• Decrypt: python vault.py decrypt secret.cev recovered.txt

• Compare secret.txt and recovered.txt to confirm that the contents match.

10. TESTING AND RESULTS

The following test cases should be executed during demonstration. Record the actual result after running the program on your computer; do not report unperformed tests as completed.

11. ADVANTAGES AND LIMITATIONS

11.1 Advantages

• Encrypted files are not readable as ordinary text.

• AES-GCM detects tampering and incorrect passwords.

• The password is not stored in the vault file.

• The design is compact and suitable for demonstrating cryptographic principles.

11.2 Limitations

• A weak password can be guessed; users should choose a long, unique passphrase.

• If the password is lost, the file cannot normally be recovered.

• The prototype reads a complete file into memory and is not optimized for very large files.

• It does not include a graphical interface, user accounts, secure backup, or key recovery.

• The prototype has not been independently security-audited and should not be treated as a production vault.

12. APPLICATIONS

Potential educational use cases include protecting personal documents, securing project files, demonstrating data-at-rest encryption, and learning password-based key derivation and authenticated encryption. Use should follow institutional and organizational security policies.

13. CONCLUSION AND FUTURE SCOPE

The Cryptographic Encryption Vault demonstrates how password-derived keys and AES-GCM can protect files against casual disclosure and detect unauthorized modification. The project covers the basic lifecycle of encryption, secure metadata handling, decryption, and error handling. Future enhancements may include a graphical user interface, chunked encryption for large files, secure key storage, access logging, encrypted backups, and a formal security review. Any extension should preserve nonce uniqueness, authenticated encryption, and careful password handling.

14. REFERENCES

• National Institute of Standards and Technology (NIST), FIPS PUB 197, Advanced Encryption Standard (AES).

• National Institute of Standards and Technology (NIST), SP 800-38D, Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC.

• Python Cryptographic Authority, cryptography documentation: https://cryptography.io/

• Python Software Foundation, Python documentation: https://docs.python.org/3/

Note: Replace the blank cover-page fields with your actual details. Run the code and insert genuine screenshots or measured results before submitting the report.

Test Case	Expected Result	Observed Result
Encrypt a valid file	An encrypted .cev file is created	To be recorded
Decrypt with correct password	Original content is recovered	To be recorded
Decrypt with wrong password	Authentication error; no plaintext written	To be recorded
Modify encrypted bytes	Authentication fails	To be recorded
Use invalid vault format	Program reports invalid format	To be recorded